

Real-world (compare your loop to the library's) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain code), **Advanced** (the differences that only show up once you compare closely). Read Basic first even if you already know LangChain — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You just built the loop by hand. Now do the exact same task, but let LangChain's own agent function run the loop for you instead. Run the same 3 questions through both, and compare: same final answers? Same number of steps?

Things to use: - `from langchain.agents import create_agent` — the current, standard way to get a ready-made tool-calling agent (this replaced the older `AgentExecutor` + `create_tool_calling_agent` pair, which are now legacy). - Wrap your existing Python function as a LangChain tool with the `@tool` decorator from Doc06. - `create_agent(model, tools=[your_tool])` — one function call builds the whole loop for you.

Intermediate Version

`create_agent` is doing exactly the loop you just wrote — the ReAct think/act/observe cycle, a step limit, tool dispatch — just packaged as one function call instead of code you wrote by hand. It's actually built on LangGraph itself (the same graph engine Doc09 teaches), so under the hood it's a small, ready-made graph. The point of this exercise isn't "which one is better," it's that having built the primitive yourself means you can now read what a library-provided agent is actually doing underneath, instead of treating it as a black box.

- `create_agent(model, tools=tools)` builds a complete, ready-to-run agent in one call — no separate "brain" object and "loop runner" object to wire together, unlike the older two-piece pattern.
- It talks in the same `{"messages": [...]}` shape as a LangGraph graph, not a plain `{"input": ...}` string — call it with `agent.invoke({"messages": [("user", question)])}`, and read the answer from `result["messages"][-1].content`.
- To set a step limit, pass `{"recursion_limit": N}` as the second argument to `.invoke()` (the same config-based limit every LangGraph graph uses, from Doc09) — there's no separate `max_iterations` constructor argument anymore.
- To see each step as it happens (the modern replacement for `verbose=True`), call `.stream(inputs, stream_mode="updates")` instead of `.invoke()` — put what it prints side by side with your own `steps` list from the previous exercise's Advanced Approach, and you should see the same shape of information, just formatted differently.

Advanced Version

Think past "does it get the same answer" — ask **does `create_agent` count a "step" the same way your loop does, and does it handle a tool's exception the same way `run_tool()` does?** One real, honest difference worth knowing: when a `create_agent`-built agent hits its `recursion_limit`, it raises a `GraphRecursionError` (from `langgraph.errors`) instead of quietly handing back a partial answer — a stricter, more "fail loudly" behavior than the older `AgentExecutor`, which by default just returned whatever it had so far. Test what actually happens when one of your tools raises an exception too, and compare it to your own `run_tool()`'s Observation-message behavior from the Intermediate exercise. Write down, in your own words, what you'd expect to be different before reading Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the pieces and the comparison to run. Intermediate explains what each piece is actually doing, mapped directly back onto the loop you already built. Advanced asks the harder question underneath the comparison — not "does it work," but "does it count and fail the same way yours does" — which is exactly what makes reading someone else's `create_agent`-based code later feel familiar instead of confusing.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
same tool, wrapped for LangChain with @tool

build the agent with create_agent(model, tools=[your_tool])

for each of your 3 test questions:
    run it through your own hand-built loop, note steps + answer
    run it through create_agent, note steps + answer
    compare
```

Here's almost the whole thing — fill in your 3 questions yourself:

```
# agent_executor_comparison_practice.py
from langchain_core.tools import tool
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    ...

llm = ChatOpenAI(model="gpt-4o-mini")
agent = create_agent(llm, tools=[get_weather])

test_questions = [...] # your 3 questions
for q in test_questions:
    my_answer = run_agent(q)
    lib_result = agent.invoke({"messages": [("user", q)]}, {"recursion_limit": 11})
    lib_answer = lib_result["messages"][-1].content
    print(my_answer, "vs", lib_answer)
```

Intermediate Version

```

wrap each Doc06 tool with @tool (a thin wrapper, not a rewrite)

agent = create_agent(llm, tools=tools)    # one call – no separate prompt/agent-bu

for each test question:
    result_mine = run_agent(question)      # your own loop, from build_rea
    result_lib = agent.invoke({"messages": [("user", question)]}, {"recursion_lim
    compare result_mine.final_answer vs result_lib["messages"][-1].content
    compare len(result_mine.steps) vs however many tool-call messages appear in r

```

`recursion_limit=11` here mirrors your own `max_iterations=5` — LangGraph counts each side of a think/act round as its own step, so a 5-round loop needs roughly `2 * 5 + 1` as its limit. Run this for all 3 questions, and write down anywhere the two disagreed, before checking the [Solution](#).

Advanced Version

Add one more comparison: make one of your tools raise an exception on purpose (same idea as the previous exercise's Advanced `run_tool()`), and run the *same* question through both. Use `.stream(inputs, stream_mode="updates")` instead of `.invoke()` to watch that failed step happen live, and compare what it shows against what your own `run_tool()` would have fed back as the Observation. Also try setting `{"recursion_limit": 3}` on a question you know needs many more steps, and compare what each one does when the limit is hit — does `create_agent` raise a `GraphRecursionError` the way your `MaxIterationsExceeded` does, or does something else happen?

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get you a working side-by-side comparison on the happy path. Advanced pushes on the 2 places most real differences hide — a tool failure, and hitting the step limit — which is exactly the kind of thing that surprises people the first time they read `create_agent`-based code in production and it doesn't behave quite like their own hand-built loop did.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)